



Chitkara

PAPER OF CSE-2025 - PYTHON PCT 25D EXAM

Q1. Compute the Gross Salary

A company having the policy of giving the perks to its executives. Company has divided the executives into four classes. The classes and corresponding perks are shown below:

Level	Perks	
	Conveyance allowance	Entertainment allowance
1	1000	500
2	750	200
3	500	100
4	250	_____

An executive's gross salary includes basic pay, house rent allowance at 25% of basic pay and other perks.

Write a program that will read an executive's class (group) number, and basic pay and then compute the Gross Salary.

Input Format

Input contains the level number, and basic pay

Output Format

basic pay + house rent allowance + perks

Sample Input

```
1
500
```

Sample Output

```
2125.000000
```

ANSWER:

Test case 1 :

Input:

```
1
500
```

Output:

```
2125.000000
```

Test case 2 :

Input:

```
2
1000
```

Output:

```
2200.000000
```

Test case 3 :

Input:

4
1000

Output:

1500.000000

Test case 4 :

Input:

3
5000

Output:

6850.000000

Test case 5 :

Input:

4
2000

Output:

2750.000000

Test case 6 :

Input:

1
2000

Output:

4000.000000

Q2. Convert temperatures from Celsius to Fahrenheit.

Rajan is working on a weather application that requires the ability to convert temperatures from Celsius to Fahrenheit. As part of this task, he needs to implement a function that takes a temperature in Celsius as input and converts it to Fahrenheit ($(\text{celsius} * 9/5) + 32$).

This feature is essential for displaying temperatures in a user-friendly format within the application.

Input Format:

The only line of input contains a single integer representing the temperature in Celsius.

Output Format:

The only line of output should print the temperature converted to Fahrenheit, rounded to two decimal places.

Code Constraints:

- The input will be a single integer (which could be positive, negative, or zero).
- The output should be formatted to two decimal places.

Example

Input

100

Output

212.00

ANSWER:

Test case 1 :

Input:

0

Output:

32.00

Test case 2 :

Input:

100

Output:

212.00

Test case 3 :

Input:

-40

Output:

-40.00

Test case 4 :

Input:

37

Output:

98.60

Test case 5 :

Input:

25

Output:

77.00

Test case 6 :

Input:

-15

Output:

5.00

Test case 7 :

Input:

212

Output:

413.60

Q3. Grade of a student

Based on the marks obtained by a student in five different subjects, calculate total marks and average of marks in five subjects. Display total marks, average and grade of student .The grade of student is displayed based on the average as follows:

Grade is A - if average ≥ 75 .

Grade is B - if average ≥ 60 and < 75

Grade is C - if average ≥ 55 and < 60

Grade is D - if average ≥ 50 and < 55 .

Grade is E - if average < 50 .

Also if total marks are zero then display message “FAILED”

Sample Input 1

```
23          //marks obtained in subject 1
78          //marks obtained in subject 2
90          //marks obtained in subject 3
45          //marks obtained in subject 4
67          //marks obtained in subject 5
```

Sample output 1

```
303         //total marks
60          //average
GRADE B     //grade
```

Sample Input 2

```
0           //marks obtained in subject 1
0           //marks obtained in subject 2
0           //marks obtained in subject 3
0           //marks obtained in subject 4
0           //marks obtained in subject 5
```

Sample output 2

```
0           //total marks
0           //average
FAILED      //grade
```

ANSWER:

Test case 1 :

Input:

35

38
43
62
22
Output:
200
40
GRADE E

Test case 2 :

Input:
65
80
90
70
60
Output:
365
73
GRADE B

Test case 3 :

Input:
50
65
45
75
35
Output:
270
54
GRADE D

Test case 4 :

Input:
48
34
35
68
95
Output:
280
56
GRADE C

Test case 5 :

Input:
0
0
0
0
0
Output:
0
0
FAILED

Test case 6 :

Input:
23
78
90
45
67
Output:
303
60
GRADE B

Test case 7 :

Input:
50
9
10
22
99
Output:
190
38
GRADE E

Q4. Inventory Shipment Tally

A warehouse tracks incoming inventory using manifests. Each manifest is a dictionary mapping **Product Code Name** to the **Shipped Quantity**. Multiple manifests often contain duplicate codes, and occasionally, quantity is logged incorrectly as text.

You must:

Calculate the Total Shipped Quantity for each Product Code.

Invalid quantity entries (non-integers) must be skipped with a warning message "**Non-integer quantity skipped**".

Output the combined product inventory.

Input Format:

A list of dictionaries: [{ Code Name : Quantity },]

Output Format:

A single dictionary with values : { Code Name: Total Quantity,}

Sample Input 1:

```
[{'PZ1': 50}, {'PZ2': 100, 'PZ1': 20}]
```

Sample Output 1:

```
{'PZ1': 70, 'PZ2': 100}
```

Sample Input 2:

```
[{'PX3': 10, 'PX4': 20}, {'PX3': 5, 'PX5': 'ten'}]
```

Sample Output 2:

```
Non-integer quantity skipped
{'PX3': 15, 'PX4': 20}
```

ANSWER:

Test case 1 :

Input:
[{'PZ1': 50}, {'PZ2': 100, 'PZ1': 20}]

Output:
{'PZ1': 70, 'PZ2': 100}

Test case 2 :

Input:
[{'A':100},{'A':200}]

Output:
{'A': 300}

Test case 3 :

Input:
[{'A':100},{'B':'x'}]

Output:
Non-integer quantity skipped
{'A': 100}

Test case 4 :

Input:
[{'A':0},{'B':0}]

Output:
{'A': 0, 'B': 0}

Test case 5 :

Input:
[{'A':50},{'A':50},{'A':50}]

Output:
{'A': 150}

Test case 6 :

Input:
[{'A':10,'B':20},{'A':'10'}]

Output:
Non-integer quantity skipped
{'A': 10, 'B': 20}

Q5. Sales Lead Generator

A sales team aggregates leads generated from various channels. Logs map **Sales Agent Name** to the **Number of Leads** generated in a week. Some logs have non-numeric entries for lead counts.

You must:

Combine the leads generated by Sales Agent Name.

If the lead count is not an integer, skip that entry and print **"Invalid lead count skipped"**

Display the final dictionary of total leads per agent.

Input Format:

A list of dictionaries: [{ Agent Name : Leads},]

Output Format:

A single dictionary with values : { Agent Name : Total Leads,}

Sample Input 1:

```
[{'Sarah': 15}, {'John': 10, 'Sarah': 5}]
```

Sample Output 1:

```
{'Sarah': 20, 'John': 10}
```

Sample Input 2:

```
[{'Mike': 30, 'Jane': 20}, {'Mike': 'thirty', 'Dave': 15}]
```

Sample Output 2:

```
Invalid lead count skipped
{'Mike': 30, 'Jane': 20, 'Dave': 15}
```

ANSWER:

Test case 1 :

Input:

```
[{'Sarah': 15}, {'John': 10, 'Sarah': 5}]
```

Output:

```
{'Sarah': 20, 'John': 10}
```

Test case 2 :

Input:

```
[{'A':100},{A':200}]
```

Output:

```
{A': 300}
```

Test case 3 :

Input:

```
[{'A':50},{A':50},{A':50}]
```

Output:

```
{A': 150}
```

Test case 4 :

Input:

```
[{'A':0},{B':0}]
```

Output:

```
{A': 0, 'B': 0}
```

Test case 5 :

Input:

```
[{'A':100},{B':x}]
```

Output:

```
Invalid lead count skipped
{A': 100}
```

Test case 6 :

Input:

```
[{'A':10,B':20},{A':10}]
```

Output:

```
Invalid lead count skipped
{A': 10, 'B': 20}
```

Q6. Factory Production Output

A factory floor generates production reports by shift. Reports map **Machine ID** to the **Units Produced**. Some units are marked "scrapped" instead of an integer count.

You must:

Sum the **Total Units Produced** for each **Machine ID**.

If the production count is **not an integer**, skip that entry and print "**Production data error, invalid unit count**".

Display the total units produced per machine.

Input Format:

A list of dictionaries: [{ Machine ID : Units},]

Output Format:

A single dictionary with values : { Machine ID : Total Units,}

Sample Input 1:

```
[{'M01': 100}, {'M02': 200}, {'M01': 50}]
```

Sample Output 1:

```
{'M01': 150, 'M02': 200}
```

Sample Input 2:

```
[{'M03': 300}, {'M04': 100}, {'M03': 'scrapped'}, {'M05': 50}]
```

Sample Output 2:

```
Production data error, invalid unit count
{'M03': 300, 'M04': 100, 'M05': 50}
```

ANSWER:

Test case 1 :

Input:

```
[{'P1': 1}, {'P2': 3}, {'P1': 2}]
```

Output:

```
{'P1': 3, 'P2': 3}
```

Test case 2 :

Input:

```
[{'A':100},{'A':200}]
```

Output:

```
{'A': 300}
```

Test case 3 :

Input:

```
[{'A':50},{'A':50},{'A':50}]
```

Output:

```
{'A': 150}
```

Test case 4 :

Input:

```
[{'A':0},{'B':0}]
```

Output:

```
{'A': 0, 'B': 0}
```

Test case 5 :

Input:

```
[{'A':100},{'B':'x'}]
```

Output:

```
Production data error, invalid unit count
{'A': 100}
```

Test case 6 :

Input:

```
[{'A':10,'B':20},{'A':'10'}]
```

Output:

```
Production data error, invalid unit count
{'A': 10, 'B': 20}
```

Q7. Package Delivery Time Audit

A courier service tracks delivery times (in minutes).

A delivery taking **40 minutes or less** is classified as "On Time". A delivery taking longer than 40 minutes is "Delayed". The day's log ends with a record of -1.

Your Task:

Process the list of delivery times.

Count the number of "On Time" deliveries (≤ 40) and "Delayed" deliveries (> 40). Stop counting when the end marker is reached.

Input Format:

A single string representing the delivery times

Output Format:

Tuple (on_time_count, delayed_count)

Note: you have to return the resultant tuple.

Sample Test case 1:

Input:

[15, 32, 28, 30, 40, -1]

Output:

(5, 0)

Sample Test case 2:

Input:

[10, 20, 45, -1]

Output:

(2, 1)

ANSWER:

Test case 1 :

Input:

[15, 32, 28, 30, 40, -1]

Output:

(5, 0)

Test case 2 :

Input:

[10, 15, 25, -1]

Output:

(3, 0)

Test case 3 :

Input:

[35, 40, 50, -1]

Output:

(2, 1)

Test case 4 :

Input:

[60, 99, 53, -1]

Output:

(0, 3)

Test case 5 :

Input:

[5, 10, 15, 20, 25, -1]

Output:

(5, 0)

Test case 6 :

Input:

[45, 60, 10, 5, -1]

Output:
(2, 2)

Q8. Fitness Challenge Reps Log

An athlete tracks their performance in a short fitness challenge, recording the time (in seconds) taken to complete a set of repetitions. A time of **60 seconds or less** is deemed "Goal Met". A time greater than 60 seconds is "Needs Improvement". The athlete signals the end of their workout data with -1.

Your Task:

Calculate the number of "Goal Met" sets (≤ 60) and "Needs Improvement" sets (> 60) from the list of times. Stop processing at -1.

Input Format:

A single string representing the completion times

Output Format:

Tuple (need_improvement_count, goal_meet_count)

Note: You have to return the resultant tuple.

Sample Test case 1:

Input:

[15, 32, 28, 30, 40, -1]

Output:

(0, 5)

Sample Test case 2:

Input:

[50, 40, 75, -1]

Output:

(1, 2)

ANSWER:

Test case 1 :

Input:

[15, 32, 28, 30, 40, -1]

Output:

(0, 5)

Test case 2 :

Input:

[10, 15, 25, -1]

Output:

(0, 3)

Test case 3 :

Input:

[35, 40, 560, -1]

Output:

(1, 2)

Test case 4 :

Input:

[60, 99, 53, -1]

Output:

(1, 2)

Test case 5 :

Input:

[75, 10, 15, 20, 25, -1]

Output:

(1, 4)

Test case 6 :

Input:

[65, 76, 10, 5, -1]

Output:
(2, 2)

Q9. Water Conservation Meter

A smart meter tracks household water usage (in gallons) during short intervals.

Usage of 30 gallons or less is classified as "**Low Consumption**". Usage exceeding 30 gallons is "**High Consumption**". The meter stops transmitting data with a -1 signal.

Task:

Process the input list of usage.

- Count the total occurrences of "Low Consumption" (≤ 30)
- Count the total occurrences of "High Consumption" (> 30)
- Stop when the list reaches the end value of -1.

Input Format:

List of integers representing water usage (ends with -1).

Output Format:

Tuple (low_count, high_count)

Note: you have to return the resultant tuple.

Sample Test Case 1:

Input:

[10, 30, 35, 20, 45, -1]

Output:

(3, 2)

Sample Test Case 2:

Input:

[14, 39, 30, 48, 21, -1]

Output:

(3, 2)

ANSWER:

Test case 1 :

Input:

[10, 30, 35, 20, 45, -1]

Output:

(3, 2)

Test case 2 :

Input:

[10, 15, 25, -1]

Output:

(3, 0)

Test case 3 :

Input:

[35, 40, 50, -1]

Output:

(0, 3)

Test case 4 :

Input:

[30, 29, 31, -1]

Output:

(2, 1)

Test case 5 :

Input:

[5, 10, 15, 20, 25, -1]

Output:

(5, 0)

Test case 6 :

Input:

[45, 60, 10, 5, -1]

Output:

(2, 2)